

FRED TALKS

4th October 2026

CONTENTS

Ways to think about token pricing

BENEDICT EVANS · 2607 WORDS

Control the ideas, not the code

ANTIREZ.COM · 1245 WORDS

Giving and taking credit in big tech companies

SEANGOEDECKE.COM RSS FEED · 1543 WORDS

Test Coverage Won't Save You

JENN COOPER · 1991 WORDS

Ways to think about token pricing

BENEDICT EVANS · 09 AUG 2026 · [SOURCE](#)

AI is in a supply crunch today, but what happens when we come out of it? How and where will supply, demand, price, capacity and capex get back into equilibrium? Today, model labs can name their price, but why won't they end up as low-margin commodity infrastructure?

There are only two things you can say with certainty about token prices: we're in a supply crunch, and this is unstable. All of the variables are in play, and the market will get shaken out over the next few years to arrive at a new equilibrium. Right now we have a lot of frantic analysis of 'time to power', but question at the end of that is whether the foundation models have sustainable pricing power, strategic leverage and value capture, or whether they become low-margin commodity infrastructure providers. At the moment, I think every dynamic we can see points to the latter.

Clearly, the situation today is transitory. On the supply side, a trillion dollars or more of data centre capex is coming down the pipe (and plenty more [semiconductor capex](#) behind that), inference efficiency continues to improve very quickly, and new models are far more (or far less!) efficient in their token use. On the demand side, although the market has been [capacity-constrained since 2022](#), the crunch in the first half of this year has been driven by sudden product-market fit in really just one use case, software development, and that's actually a pretty small field (imagine if we had product-market fit for a consumer use case with hundreds of millions of DAUs - today's infrastructure couldn't support it at any price). We don't know what the next use-cases to scale will be, nor when that would be, nor what their token needs would be.

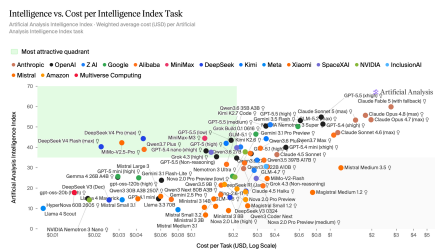
Going up a level, it's been pretty widely reported that inference today has 40-50% gross margins: this includes depreciation of the associated server costs (or the cost of renting them), but we don't really know the asset life (five years? Seven years?) and obviously this doesn't include the cost of training the next model a couple of times a year, which is currently far larger than revenue. In principle, inference is a marginal cost and training is a fixed cost, so with high enough revenue you can reach profitability, but we don't know how training costs will change. On the other side of the table, it's unclear how much of the surge in use in the last few months has an ROI (or at least has an ROI that can be quantified to a CFO), let alone any future use cases, and hence what prices people might be prepared to pay for them.

So, all the variables will move all over the place over the next 12 months, and move again over the next three to five years. How could we suggest where this will settle? How and where will supply, demand, price, capacity and capex get back into equilibrium?

In theory, you can model this bottom-up. You can make some assumptions around each of the variables I suggested above, and then try to model out how many chips there are now, how many chips with what performance TSMC and the rest of the semis industry might be able to deliver when, and how fast all of that can be brought online in data centers and how fast those can be powered. Then you can wonder about price discipline, and make some guesses about use cases. This will get you a number, but it will be rather like trying to build a five-year forecast for the broadband market in 1998: the spreadsheet will be very pretty, and you might even get close to the right number for this year, but there are too many unknown variables to make a useful forecast of a longer-term market structure.

In other words, we can say that token price is a function of supply and demand at a level between the sellers’ marginal cost and the buyers’ ROI, but we don’t actually know what supply, demand, marginal cost or ROI will be.

The other approach is to look from the top down: how do things like this tend to play out? What are the building blocks, and where can they go? Most of this conversation depends on what happens to this curve.



First, how many people will pay to be at the top right of the curve - to be at the frontier? At one extreme there are already use cases that already work just fine with a small, old, perhaps open source model that runs for ‘free’ on-prem or on your phone; at the other extreme there will be some that get better results from the latest, most expensive frontier model, consuming lots of tokens for lots of money; and then there will be many that are somewhere in between. So, how many use cases get better results from going how far up the cost curve, and how many have an ROI for that, and how much of the use is handled by models that are smaller, cheaper, good ‘enough’ and much more commoditised? The Panglossian view is that ROI might go *up* with more expensive frontier models because they have better results, but where does that really apply?

Second, does the frontier keep moving significantly? This is obviously the most basic science question in AI: how long does the frontier keep getting better, how long does that keep needing more and more compute, and does that continue to happen at a rate that keeps it ahead of downward pricing pressure from efficiency and capacity gains? Does the expensive head of the curve continue to be a thing?

Third, will there still be fierce competition between frontier models? Does the field shrink to fewer and fewer frontier models, perhaps with network effects emerging? Do frontier models diverge, with different models having much clearer leads in different fields? That could be another path to sustainable pricing power. Or do we continue with a mid-single-digit number of companies that are all making frontier models that all have generally equivalent capabilities? At the moment, everyone is using mostly the same science and mostly the same training data, and getting mostly the same results, and we don't yet know of a network effect or any other winner-takes-all effect that would let one company pull ahead, stay ahead, and do things that others could not, in some sustainable way. Does that change?

Fourth, how much of the value from those high-end use cases is captured by the frontier model itself? How much needs to be wrapped in tooling, process, proprietary data, go-to-market, networks, support, and everything else associated with a traditional software company, even if you do need the big expensive frontier model underneath? Can that model do the whole thing, or is the model, no matter how good, still a piece of infrastructure that you use to make the actual product? At the extreme, can the model itself invent and make all of those things, and would that let them charge by seat, by outcome or just take the profit? Or do even (or especially) the most sophisticated and high-value use-cases need to sit inside hundreds of new companies that can pick and choose which models to use?

None of these are binaries: they're all a question of degree, and they'll probably vary quite a lot by use case. But at one extreme, there are two or three giant minds that run half of everything and have massive pricing power, and at the other extreme LLMs look like databases - there'll be millions of them, some very big and some very small, and the value is in what you build on top - after all, every SaaS company is a 'database wrapper'. There's a future in which Anthropic (or a company we haven't heard of yet) wins the whole thing and can set its own terms, and a future in which dozens of routers run real-time auctions to allocate your tasks across hundreds of low-margin model-farms and a benchmark company takes a fee on every single one.

I don't think anyone can actually *know* the answer yet. I've said "we don't know" a lot, and that's very deliberate. Part of the concept of the 'S Curve' is that there's a stage early in the emergence of a new technology where it's clear that this is going to be huge but nothing else is clear at all - the mid 1990s for the internet, say, and 2008 or 2009 for mobile. There are places where you can take a view - for example, I've argued at length that I think chatbots are a poor

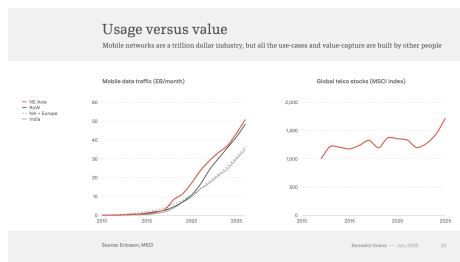
interface that will struggle to capture value up the stack - but we should presume there are big questions that we can't see yet, let alone answer, and anyone picking one of the ten possible outcomes we can see and saying "it will be that one!" is just guessing.

Meanwhile, there is structural uncertainty at the early stages of every big new technology, but the uncertainty now is different, because we don't have a good theoretical understanding of why these models work so well and so we don't know how much better they can get. In 1995, we didn't know how the internet would evolve but we knew that there were less than 100m PCs on earth (and they were expensive) and that telcos couldn't give everyone FTTH next year; in 2010 we didn't know what the next iPhone would be but we knew it wouldn't have retinal projection. We knew the physical limits in ways we don't really know with LLMs. Next month a new approach could cut inference compute needs by 90%, or double demand, or both.

All of this has people hunting for patterns to recognise (indeed, it's been observed that *all* conversations about AI end in a hunt for metaphors). It has become common to draw comparisons with fiber, which had a massive overbuild in the Dotcom bubble that looks a bit like the infrastructure build-out today. The narrow problem with that is that fiber construction was far ahead of demand, where AI compute construction is far behind demand, though, as above, we don't know what that supply/demand balance will look like in the future. But the more relevant objection, I think, is that fiber construction was mostly fixed cost (digging holes) rather than marginal cost (more equipment), whereas growth in compute needs means you need to buy more compute.

That makes mobile data a more fruitful comparison here. Mobile networks have marginal cost for capacity, and like AI they had an enormous surge in usage 15 years ago, that overwhelmed capacity and had carriers scrambling to add capacity and rebalance their pricing. Meanwhile, selling bits looks superficially similar to selling tokens: it's an opaque measure of marginal cost that doesn't map in any transparent or intuitive way to use cases or value, and needs to be replaced with bundles of some kind. But most importantly, in the last 20 years cellular data traffic has risen by several orders of magnitude, and this has become an enormous industry, with annual revenue of a trillion dollars and capex of \$200 billion, but the stocks have gone nowhere, and all the value was captured by other people further up the stack. This, of course, is one of the core questions for AI: is this going to be low-margin commodity infrastructure with all the value captured by other people further up the stack?

[View fullsize](#)



Semiconductor manufacturing also has echoes of AI, since unlike cellular it has something of the same escalating cost and complexity that we see in foundation models. Rock's Law pointed out that the cost of a cutting edge fab doubles every four years: over time the frontier of semiconductors become so hard and so expensive that the number of players dropped from dozens to a handful and now really only one, TSMC. That's another core question for AI: will this get so hard and so expensive that only a couple of people can do it, even without network effects? Equally, many semiconductor uses are further back along the same kind of price/performance curve I just discussed for AI. Even here, though, while TSMC has a *de facto* monopoly on the frontier, and nice margins, it doesn't actually capture a large share of value from the broader tech economy: net income last year was \$53bn, less than half of Apple alone.

There are plenty of other comparisons one could make here: in the last six months Sam Altman has compared OpenAI both to Windows, a high-margin capital-light monopoly based on network effects, and electricity utilities, which are natural monopolies but also low-margin regulated utilities selling a pure commodity. One could also point to cloud (three leading players with good margins and clearly distinguished propositions, but again limited value-capture). But none of these have predictive value: analogies don't have predictive value. You can't prove whether something will have the same outcome as mobile by arguing how much it's like mobile: this was a mistake that many very clever people in tech made 15 years ago, claiming that Android would beat iOS because Android was 'open' and 'open' Wintel had beaten the 'closed' Mac in the 1990s (it's also the mistake that doomers make when they claim that AI is 'like' nuclear weapons). Everything is different: bits, tokens and transistors are different, each of these examples are different from each other, and AI will be different too.

However, these examples do tell us, empirically, that something can be very important, very expensive, change the world, and be full of very sophisticated science and engineering, and yet have a wide range of possible outcomes. There isn't one inevitable path here: you can have price equilibrium at high margins and at low margins, and with and without market concentration, and you can't hand-wave that away by talking about AGI and saying "you don't understand exponentials!"

However, if one thread in everything I've written above is how much we don't yet know, the other thread is that every path to foundation models having market dominance, strategic leverage, value capture, winner-takes-all effects, or anything else other than becoming commodity infrastructure, requires something to change.

Maybe frontier models will become less competitive - yet in the last six months, Mark Zuckerberg and [Elon Musk](#) jumped from zero back onto the leaderboards. Maybe network effects will emerge. Maybe chatbots can grow into products and don't need to be wrapped in software. Maybe one lab will start out-executing all the others and pull ahead on sheer product dynamism - Microsoft, Google, Facebook and Apple had to execute their way into a winning position before they had winner-takes-all efforts. Maybe *something* else will happen.

In particular, we have not one but two potential *dei ex machina* - Trump and China. China is reportedly [considering](#) regulating open source and some people close to Trump have floated this as well (though since Meta abandoned Llama the US has no leading open model), and export controls could expand and become systematic. Many people see the pleas for regulation from Anthropic and (sometimes) OpenAI as a front for regulatory capture, but either way, we can't presume this will remain an entirely free market.

Even so, that brings me back to the same point: the current market dynamics point to a future in which, as today's supply crunch eases, frontier models move towards becoming commodity infrastructure, with all of the value built on top, and for a different outcome, something needs to happen that we don't see yet.

Control the ideas, not the code

ANTIREZ.COM · 18 JUL 2026 · [SOURCE](#)

Look at the past history of this blog. There are many blog posts about

So mine is a trick. People feel more and more programming is complete

This is why yesterday, on X, I said that I believe many programmers a

1. You can now generate a lot of code, even **not** accounting for the

2. LLMs are very good at writing locally optimal code, and are worse

3. The working day is 8 hours. If you read the code, it is a tradeoff
Controlling the ideas. Do you remember this phrasing from the Mythica
Matteo Collina yesterday asked me, in reply to my tweet: but didn't y
Fable and GPT 5.6 reviews to the sorted sets memory saving are going
:

Giving and taking credit in big tech companies

SEANGOEDECKE.COM RSS FEED · 02 AUG 2026 · [SOURCE](#)

Engineers often complain that visibility should be their manager's job. In other words, they think engineers should be able to focus on the code, while their manager figures out who's doing well and rewards them.

This attitude is an extension of the “school fantasy”: the idea that your workplace should operate by the same rules as your school or university. After all, you didn't have to worry about “visibility” during your education. You simply did the assignments and tests you were given, and if you did well you were rewarded with a good grade.

Many big tech companies encourage this attitude, because it helps them recruit smart graduates. They fashion their workplaces to look and feel like a university, even calling the physical space “campuses”. But it's still work, not school. If you treat it like school, you are going to have a bad time.

Taking credit

The first lesson many new engineers learn is that **you have to take credit for your work**. If you silently jump in to help a struggling project and get it back on track, there's no guarantee of reward. Credit will naturally flow to the project lead, not you. In fact, if this project is outside of your direct team, it's likely you will be *punished* for it: to your manager, it will look like you're simply doing nothing at all.

Even when your manager is watching your work, credit is largely uncorrelated with how well you did. That's because, unlike at school, **you are the subject-matter expert on your own work**. Software systems are so complicated that only the people who work on them can hope to understand them, and even that understanding is always imperfect. If even experts can't reliably estimate the difficulty of changes, how is your manager supposed to assess your technical performance? The answer is they aren't. They're simply not qualified to assess it.

Instead, smart managers will find engineers on your team they trust and ask them how you're doing. On small teams that have worked on a single codebase for a long time, this works okay, because everyone's familiar enough to judge everyone else's work. On large teams with a high rate of codebase churn, it goes badly, since they're just guessing. On teams with a nasty, cutthroat culture, it sometimes goes *very* badly, since this is a good opportunity to actively sabotage the engineers who might threaten you.

Experienced engineers know how to **take the credit themselves**. When they do something good, they tell their manager about it. They write internal posts explaining why it was technically difficult and how they solved it (the audience for these is partially those trusted engineers, and partially the managers who will see a long technical post and think "wow!" without reading it). They actively build trust with their management chain. Worrying about this stuff is the beginning of playing politics.

Giving credit

There's a kind of engineer who's learned how to take credit but hasn't learned any other lessons yet. They're proactive about telling people what they've done, and they always maintain a "brag doc". In particular, they love to talk about the parts they did *by themselves*, since those are least vulnerable to other people coming in to claim credit. You can tell they're jealously guarding whatever credit they've managed to accumulate. The lesson this kind of engineer hasn't learned is that **you can often accumulate credit best by giving it away**.

To see why, consider how credit flows *up* inside a tech company. I wrote above that your manager can't assess the quality of your technical work on their own, but instead has to rely on other engineers they trust. They'll quietly ask those engineers "hey, was this project really that impressive?". In fact, often there are multiple layers of this at play¹. In big companies, line managers usually don't decide who gets promoted or who gets a raise: they make recommendations to their manager, who has their own network of trusted engineers (confusingly, sometimes these networks overlap). The point is that **there is a large group of people behind the scenes who will quietly and informally judge the value of your work**.

Succeeding at a tech company is largely about finding ways to get these people on your side. The easiest way is to share your credit with them — and since you don't know who exactly is in this group, you should be sharing your credit freely. When you get feedback from other engineers, publicly thank them and mention them in your internal posts about the project. Find opportunities to ask for small favors, so you have an excuse to give other people credit. As best you can, make your individual projects at least partially *group* projects.

Sharing credit with others gives them a reason to support you. A shared project you've worked on reflects well on everybody: on you, for working well with others, on the people you've worked with, for the same reason, and for your manager, for fostering such a great environment of cooperation. Lots of people have good reason to talk that project up, because it's partly their project too. On the other hand, a project you've jealously kept to yourself reflects well on nobody: you come across as antisocial and your peers come across as unhelpful.

Blame

Blame operates by the same rules as credit. When something goes badly wrong, managers will ask their networks “hey, who screwed up here?” The answer to this question is never simple. Even on a purely technical level, failures always involve an interaction between multiple complex systems, any one of which could conceivably have been built so as to avoid the failure. In other words, **competent engineers can assign blame pretty much wherever they want.**

Because of this, it's risky to have a project for which you're clearly the only one getting credit. When something goes wrong, the network of people who will assign blame will likely be implicated in every part of the system but yours. They will be incentivized to attribute fault to the brand-new thing that they don't understand and are not responsible for. If instead that network had been involved in your project — if they'd been in a position to share the credit — they'd be less incentivized to blame it.

Of course, engineers are (mostly) not scheming viziers who make purely self-interested decisions. When asked who to blame, they usually make a good-faith effort to answer honestly. But in an area where there's no single clear right answer, it's human nature to be at least a little bit guided by your incentives. Nobody likes to think they're responsible for a group failure.

Conclusion

Credit and blame are the currencies of tech companies (and often directly translate to the actual amount of currency you get to take home). For technical roles, managers assign credit and blame based on lots of quiet conversations with their trusted engineers. This can be a rude awakening for very junior engineers who are used to having their work assessed by an expert grader (or less junior engineers who haven't yet shaken that mindset completely).

Don't expect to get credit simply by putting your head down and doing good work. You have to find some way to tell people what you're doing and why it's important: internal blog posts, mentioning it in 1:1s with your manager, or anything else you can think of. But don't take self-promotion too far. It's a bad idea to try and hoard all the credit for your projects, for two reasons.

First, sharing credit with other people gives them a reason to talk positively about your project. Credit is not a zero-sum game: if you do it right, you can get other people to build up your credit for you. Second, hoarding credit sets yourself up as a lightning rod for blame. Projects where the credit is concentrated in one or two people are automatically² blamed for complex problems, because nobody is incentivized to defend them.

1. This is a classic example of an illegible-but-essential part of a software company. I wrote about this general phenomenon in *[Seeing like a software company](#)*.

↩

2. Of course, if you do really screw up, you'll be blamed no matter what. I'm talking here about complex failures where it's non-trivial to attribute blame to a single source.

↩

Test Coverage Won't Save You

JENN COOPER · 09 JUN 2026 · [SOURCE](#)

A year ago, if a codebase accumulated three different ways to do the same thing, somebody would usually notice.

A reviewer might leave a comment. A senior engineer would mutter something about "yet another helper," and eventually the team would clean it up. Maybe they'd write a short doc. People would learn, collectively, that this was not the way.

This was not a perfect system. Human review can be slow. People miss things. Pattern docs get stale. Engineers develop opinions that are 70% wisdom and 30% scar tissue.

Still, the friction did something useful. It kept codebases from drifting too quickly.

Coding agents change that.

A room with too much electricity.

"Now, this is a room with electricity. But it has too much electricity."

Once agents are writing a meaningful share of code, the question is no longer just "can we get working code faster?" Obviously, yes. We can. Sometimes hilariously so.

The more interesting question is: what does the codebase teach the next agent?

Because it does teach.

Every merged PR becomes a precedent. If the repository contains one clear pattern, the next agent has a decent shot at following it. If it contains three slightly different patterns, the next agent may extend one, combine two, or invent a fourth for reasons that are, in technical terms, vibes.

The funhouse builds itself

Coding agents, like some of us, have a chaotic inner goth teenager.

Their non-deterministic nature means they can be inconsistent, often in surprising ways. They can generate the weird little leap that solves the problem, or they can generate novelty where nobody asked for it.

The dangerous thing is that most of the code is fine, if you look at it in isolation.

A PR passes tests. The implementation makes sense. The file is clean enough. You look at the diff and think, yeah, sure, that seems okay.

But zoom out, and the codebase as a whole has started to get weird.

- three near-duplicate utilities
- multiple data-fetching patterns
- parallel abstractions solving almost the same problem

The codebase remains locally **cohesive**, while slowly losing global **coherence**.

That matters more than it used to, because even more than humans, agents are responsive to the signals around them. A clear codebase gives them clear precedent. A muddled codebase gives them confusion.

You can put principles in **CLAUDE.md** or docs, but they're... advisory. They compete with task instructions and inline comments for attention, get summarized when context windows compact, and depend on the agent attending to the right instruction at the right moment.

If your codebase contains three slightly different ways to solve a problem, the next agent has to infer which one is canonical.

From the agent's point of view, this is not irrational. The repository gave it mixed evidence.

The scary outcome is not code that obviously fails. It is code that superficially keeps working, while the shape of the system increasingly gets worse.

Tests are nice

Most discussions about AI-native development jump from this problem – agents' tendency to accumulate tech debt – directly to tests.

And yes, across the industry, teams are writing dramatically more tests than they ever have. Agents have made high test coverage affordable in a way it never used to be.

Recently, Garry Tan argued that the primary way to keep AI agents on track is 90% test coverage:

| *Tests are the ratchet. 90% coverage, every PR, no exceptions.*

And indeed, test coverage is the simplest kind of ratchet: a mechanism that allows motion in one direction only, like a socket wrench that turns the bolt forward but never lets it spin back. Once a test locks in a behaviour, it becomes difficult to accidentally regress.

But it's worth being specific about what tests actually do.

Unit tests check that a function still returns what it returned before. Integration tests check that pieces still wire together the same way. E2E tests reach further: they check whether the product still does what it used to, and the assertions you prioritize there should be an important act of human judgment.

But they all share the same basic shape: tests verify that code does what it did before.

Whether what it did was even the *right way to do it* is a separate question.

Tests ratchet behavioural sameness.

Meanwhile docs and evals can pin down reasoning and behaviour bars, which is sometimes more important. But none of these is directly looking at the *shape* of the system.

90% coverage of good patterns

A coverage ratchet only improves the codebase if the patterns it's locking in are already good ones.

If a questionable pattern already exists in the code, the next agent is more likely to extend it. The tests generated around that implementation only reinforce it further.

Over time, high coverage can unintentionally fossilize architectural drift, rather than prevent it in the first place.

At first, especially during prototyping, this can be easy to miss, because nothing looks obviously broken. Coverage stays high. The system works. But over time, certain parts of the codebase become strangely resistant to cleanup.

Tests that protect the wrong behaviour create drag against coherence. The harder you push for coverage, the more deeply you can lock in the shape you happened to start with.

This isn't a criticism of tests. Tests are necessary. But tests primarily validate behaviour, while many of the emerging failure modes in agent-generated systems, especially as they grow, are failures of *shape*.

A load-bearing poster.

"Uh, I wouldn't take it down if I were you. It's a load-bearing poster."

Fitness functions protect the shape of the system

Unit tests ask whether the code still does what it did before. Fitness functions ask a different question: is the system still shaped the way we want?

Most codebases already have a few simple versions of this. Linters, type checks, and import-boundary rules all encode some idea of acceptable shape, at the micro level.

But in agentially-maintained codebases, we need an additional kind of macro fitness function. It has a couple of important jobs:

- catch bad agent choices at the moment of work
- redirect agents with the right amount of friction for the violation

At Forestwalk, for example, we have a fitness check that prevents tool handlers from calling back into our AI services. Tool handlers run inside the agent's own loop, so any handler that kicks off more inference can stack model calls inside model calls and freeze a live meeting. Agents sometimes accidentally introduce this pattern, so our fitness check flags it and ensures it's not committed.

Other fitness tests are ratchets that apply pressure to measurable properties – heuristics for shape like file size, bundle growth, prompt size, import fanout, and dependency count.

For these, the ceiling starts at the current value (e.g. 837kb) and flags to agents and humans if a PR would unduly increase it. For instance, when an agent adds an unnecessary new dependency that blows up our frontend bundle, the check fails.

The important part isn't just the limit itself. It's the “just in time” agent coaching.

At first, we found agents would sometimes over-optimize against whatever metric the checks exposed, with great resourcefulness and in ways we did not want. But refining the flags to give common sense guidance helped steer the agents into considering tradeoffs correctly.

For example, an agent might see this just in time:

🔴🔴🔴🔴🔴🔴 Failed Checks: 1 🔴🔴🔴🔴🔴🔴

```
FAIL src/frontendBundleBudget.test.ts > Frontend bundle size budget
AssertionError: Frontend bundle grew +29 KiB (700 -> 729 KiB; fails a
```

This ratchet is review pressure, not a mandate to shrink the number a

Diagnose first:

- Is the growth accidental (dependency, duplicate/dead code, broad im
- Does the added JS unlock user-visible value?
- Can cleanup reduce it without hurting UX, accessibility, tests, or

Quality-preserving fixes:

- remove duplicated rendering paths or dead production code
 - move repeated styling or static configuration out of lazy-loaded Ja
 - lazy-load rarely used flows instead of loading them on the main pat
- (more examples here)

Do not fix this by:

- omitting requested scope or deferring needed UI work
 - removing aria-label/title text or visible affordances
 - deleting helpful empty/error-state copy
- (more examples here)

For intentional growth, (procedure here) and explain why in the PR.

If you simply tell agents to "reduce bundle size," they will sometimes do naive things like:

- delete empty-state copy
- replace styled controls with browser-native ones
- inline dependency code to avoid import costs
- remove accessibility affordances

Instead, the idea is to not just explain what a given fitness test is protecting, but also explain what is *not allowed* to be sacrificed in the process, and do so at the moment the agent is working.

In contrast, the same guidance in `CLAUDE.md` is often skimmed past or applied inconsistently by agents. A just-in-time check, on the other hand, creates desired friction between the agent and a finished task. Where humans get tired and occasionally creative about routing around friction like this, agents are quite willing to schlep through feedback and loop until the check is passing.

A master bedroom.

"Come on in, it's your master bedroom!"

What the checks can't see

Fitness functions are good at catching things precise enough to encode.

That is useful, but coherence is not always that crisp. Some drift is obvious only in relation to the rest of the codebase: this new thing is too close to an existing thing, this test is protecting a pattern we no longer want, this change is making the next change harder.

So in addition, we use a few non-deterministic checks on top of the deterministic ones.

One runs before a PR is opened. It scans the proposed change against the existing codebase, and asks whether the agent has introduced or expanded a competing pattern. When it finds one, it points to the existing pattern, explains why the new one looks redundant, and asks the agent to either extend the original or justify why this case is genuinely different.

Most of the time, the agent course-corrects quickly.

We run a similar check again at the PR review level, where it has more context and can catch subtler pattern drift.

Another check runs nightly and looks for tests that have fossilized patterns we have since decided against. Its mission is to find tests that protect old implementation choices that, if left alone, would teach future agents to preserve patterns we no longer want.

These checks are not magic. They miss things. They can be noisy. They still need human judgment around them.

But the goal is not to eliminate judgment. The goal is to move more human judgment into places where it can be highest leverage.

Making coherence the easy path

Fitness ratchets are not the whole answer to maintaining coherence in agent-generated codebases.

Fitness functions are one layer. Pattern drift checks are another. Product verification, code review, evals, documentation, repository structure, and the actual taste of the humans steering the system all matter too.

However, it's worth investing regularly in the early layers: the ones closest to the moment an agent is making an implementation choice.

That is where we have found these types of fitness functions useful. They do not solve coherence by themselves, but they add friction in the right places. Bad patterns get interrupted earlier. Good patterns have a better chance of becoming precedent.

The same feedback loop that spreads bad patterns can be used to spread good ones.

So there's hope. When you see this pattern work, it soothes some of the worry we all have about agents causing chaos. They're a lot more useful when the codebase gives them an obvious next move. The right pattern is easier to find. The weird fourth version of the thing has a harder time looking reasonable.

It's not perfect. Keeping a growing codebase coherent still requires human judgment, architectural intuition, and social coordination. The system still needs to be built.

But that system, itself, increasingly feels like the work.

If the codebase trains the next agent, then the job is not just to write better code.

It is to build the environment where better code is the obvious thing to write.